

02AML204 – INTRODUCTION TO ARTIFICIAL INTELLIGENCE

SLE-2: PROFILING REPORT

Empirical Performance Analysis of BFS and DFS

PRN: 25UAM095

Name: Arpita Taradale

Division: B

Date: 22-09-2026

GithubLink:

1. Algorithms / Versions Profiled

Algorithm A: Breadth-First Search (BFS)

Algorithm B: Depth-First Search (DFS)

Problem Used:

A small 15-node graph was used for the experiment. The starting node was **A** and the goal node was **M**. The same graph, starting node, and goal node were used for both BFS and DFS to make the comparison fair.

Both algorithms successfully found the path:

A → D → I → M

2. Profiling Method

The performance of BFS and DFS was measured using actual execution-time measurements and node counting.

Tools Used:

- Python timeit module for average execution time
- Manual node counter for nodes expanded
- py-spy 0.4.2 for profiling and flamegraph generation

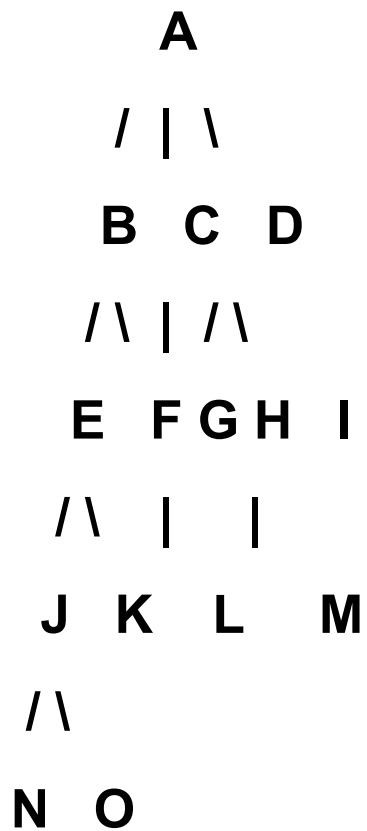
The BFS and DFS algorithms were executed on the same 15-node graph. The profiling program calculated the average execution time for each algorithm. Manual counters recorded the number of nodes expanded.

Separate py-spy profiles were also generated:

- bfs_profile.svg
- dfs_profile.svg

The py-spy runs completed with **0 errors**.

Graph:



Start node: A

Goal node: M

Path used by both: A → D → I → M

3. Results

Measurement	BFS	DFS
Average execution time	4.7723×10^{-6} seconds	7.7606×10^{-6} seconds
Nodes expanded	13	15
py-spy profile	bfs_profile.svg	dfs_profile.svg
py-spy samples	8	7
Errors	0	0

Path Found

BFS: $A \rightarrow D \rightarrow I \rightarrow M$

DFS: $A \rightarrow D \rightarrow I \rightarrow M$

Short Observation

Both BFS and DFS successfully found the same path from A to M. BFS expanded **13 nodes**, while DFS expanded **15 nodes**. The measured average execution time was approximately **4.77×10^{-6} seconds for BFS** and **7.76×10^{-6} seconds for DFS** in this experiment.

4. Justification & Analysis

BFS and DFS were tested on the same 15-node graph with the same starting node A and goal node M.

Therefore, the experimental conditions were kept the same for both algorithms. BFS expanded 13 nodes, while DFS expanded 15 nodes. The measured average execution time was also lower for BFS in this particular experiment. The results show that the search order can affect the number of nodes explored and the measured runtime. py-spy successfully generated separate profiling files for both algorithms with zero errors. Since the graph is small and the execution times are very short, the measured timing should be interpreted as specific to this graph, implementation, and computer environment rather than as a universal result for BFS and DFS.

5. AI Contribution Note

AI Tool Used: ChatGPT

What AI helped with:

- Understanding BFS and DFS
- Preparing and organizing the profiling experiment
- Explaining py-spy commands
- Preparing the profiling code

- Structuring the SLE-2 report
- Explaining the profiling results

What I did myself:

- Created and maintained the GitHub repository
 - Implemented and executed the BFS and DFS programs
 - Created the 15-node graph
 - Installed and used py-spy
 - Ran the profiling programs
 - Collected the actual execution-time results
 - Verified the paths and node counts
 - Generated and checked the BFS and DFS profiling files
 - Pushed the final project to GitHub
-

1)BFS

```
C:\Users\Arpita\AppData\Local\Python\pythoncore-3.14-64\Scripts\py-spy.exe record --output bfs_profile.svg -- python bfs_only.py
```

find_and_load (<frozen importlib_bootstrap:1371)	find_one (__init__.py:87)	find_one (__init__.py:90)
find_and_load_unlocked (<frozen importlib_bootstrap:1342)	find_and_load (<frozen im...	load_default_config (conva...
load_unlocked (<frozen importlib_bootstrap:938)	find_and_load_unlocked (<...	__init__ (commands.py:1029)
exec_module (<frozen importlib_bootstrap_external:1061)	load_unlocked (<frozen im...	__init__ (commands.py:442)
__call__ with frames removed (<frozen importlib_bootstrap:491)	<module> (commands.py:5)	load_config (config.py:64)
<module> (__init__.py:8)	find_and_load (<frozen im...	load_global_config (config...
find_and_load (<frozen importlib_bootstrap:1371)	find_and_load_unlocked (<...	load_one_config (config.py...
find_and_load_unlocked (<frozen importlib_bootstrap:1342)	load_unlocked (<frozen im...	resolve_config (config.py...
load_unlocked (<frozen importlib_bootstrap:938)	<module> (config.py:1)	resolve_config (config.py...
<module> (logging.py:1)	find_and_load (<frozen im...	is_valid_url (config.py:4...
find_and_load (<frozen im... __init__ (logging.py:119)	find_and_load_unlocked (<...	find_and_load (<frozen im...
find_and_load_unlocked (<... supports colour (logging.py:100)	load_unlocked (<frozen im...	find_and_load_unlocked (<...
find_and_load (<frozen im... find_and_load (<frozen importlib_bootstrap:1371)	exec_module (<frozen impor...	load_unlocked (<frozen im...
exec_module (<frozen impor... find_and_load_unlocked (<frozen importlib_bootstrap:1342)	__call__ with frames removed ..	
load_unlocked (<frozen impor... load_unlocked (<frozen importlib_bootstrap:924)	<module> (__init__.py:106)	
__call__ with frames removed ..	find_and_load (<frozen im...	
module_from_spec (<frozen importlib_bootstrap:816)	find_and_load_unlocked (<...	
create_module (<frozen importlib_bootstrap_external:1054)	load_unlocked (<frozen im...	
__call__ with frames removed (<frozen importlib_bootstrap:491)	exec_module (<frozen impor...	
	__call__ with frames removed ..	
	<module> (decoder.py:5)	
	handle_fromlist (<frozen ...	
	__call__ with frames removed ..	
	find_and_load (<frozen im...	
	find_and_load_unlocked (<...	
	load_unlocked (<frozen im...	
	exec_module (<frozen impor...	
	__call__ with frames removed ..	
	<module> (scanner.py:13)	
	or_ (enum.py:1593)	
	__call__ (enum.py:709)	
	new_ (enum.py:1179)	
	missing_ (enum.py:1513)	
	iter_member_by_def_ (enum...	

2)DFS

```
C:\Users\Arpita\AppData\Local\Python\pythoncore-3.14-64\Scripts\py-spy.exe record --output dfs_profile.svg -- python dfs_only.py
```

find_and_load (<frozen importlib_bootstrap:1371)	find_one (__init__.py:87)
find_and_load_unlocked (<frozen importlib_bootstrap:1342)	find_and_load (<frozen im...
load_unlocked (<frozen importlib_bootstrap:938)	find_and_load_unlocked (<...
exec_module (<frozen importlib_bootstrap_external:1061)	load_unlocked (<frozen im...
__call__ with frames removed (<frozen importlib_bootstrap:491)	<module> (commands.py:5)
<module> (idliback.manager:3)	find_and_load (<frozen im...
__init__ (idliback.manager:3)	find_and_load_unlocked (<...
__getattr__ (<frozen impor... find_and_load_unlocked (<frozen importlib_bootstrap:1342)	load_unlocked (<frozen im...
find_and_load (<frozen im... load_unlocked (<frozen importlib_bootstrap:938)	<module> (config.py:1)
find_and_load_unlocked (<... <module> (logging.py:1)	find_and_load (<frozen im...
load_unlocked (<frozen im... find_and_load (<frozen importlib_bootstrap:1371)	__init__ (logging.py:119)
exec_module (<frozen impor... find_and_load_unlocked (<frozen importlib_bootstrap:1342)	supports colour (logging.p...
find_and_load (<frozen impor... load_unlocked (<frozen importlib_bootstrap:938)	find_and_load (<frozen im...
get_code (<frozen zipimpor... exec_module (<frozen importlib_bootstrap:1179)	__call__ with frames removed ..
get_module_code (<frozen ... <module> (<frozen os:1123)	load_unlocked (<frozen im...
get_data (<frozen zipimp... <module> (<frozen os:1123)	module_from_spec (<frozen ...
PathLike (<frozen os:1140)	find_and_load (<frozen importlib_bootstrap:1371)
	find_and_load_unlocked (<frozen importlib_bootstrap...
	load_unlocked (<frozen importlib_bootstrap:938)
	__call__ with frames removed ..
	exec_module (<frozen impor...
	__call__ with frames removed ..
	<module> (decoder.py:3)
	find_and_load (<frozen im...
	find_and_load_unlocked (<...
	load_unlocked (<frozen im...
	exec_module (<frozen impor...
	__call__ with frames removed ..
	<module> (__init__.py:125)
	find_and_load (<frozen im...
	__enter__ (<frozen importl...
	acquire (<frozen importlib...
	__enter__ (<frozen importl...
	setdefault (<frozen import...

6. Conclusion

This profiling exercise demonstrated how empirical performance analysis can be used to compare search algorithms. BFS and DFS were tested on the same 15-node graph using the same start and goal nodes. Both algorithms found the required path. BFS expanded 13 nodes compared with 15 nodes for DFS and recorded a lower average execution time in this particular experiment. The use of timeit, node counting, and py-spy provided practical performance measurements instead of relying only on theoretical analysis.

7. GitHub Repository

GitHub:

<https://github.com/ArpitaTaradale/BFS-DFS-Performance>

Profiling Files

- bfs_profile.svg
- dfs_profile.svg

Main Python Files

- bfs_dfs.py
- bfs_only.py

- dfs_only.py
 - profiling.py
-

8. Final Profiling Evidence

BFS:

- Average Time: 4.7723000170663e-06 seconds
- Nodes Expanded: 13
- py-spy Samples: 8
- Errors: 0

DFS:

- Average Time: 7.76059995405376e-06 seconds
- Nodes Expanded: 15
- py-spy Samples: 7
- Errors: 0